

A thick dark blue vertical bar runs down the left side of the page. At the bottom of this bar, several thin, curved lines in dark blue and light grey sweep upwards and to the right, creating an abstract, organic shape.

PROGRAMACIÓN
Tema 1

ALGORITMOS

ÍNDICE

DEFINICIÓN DE ALGORITMO.....	3
ELEMENTOS PARTICIPANTES EN LA REALIZACIÓN DE UN ALGORITMO.	3
FORMAS DE DESCRIBIR UN ALGORITMO.....	4
A. LENGUAJE NATURAL.....	4
B. ORGANIGRAMAS.....	5
C. PSEUDOCÓDIGO.....	6
D. LENGUAJES DE PROGRAMACIÓN.....	7
DESARROLLO E IMPLEMENTACIÓN DE UN ALGORITMO.....	8
CONCEPTOS FUNDAMENTALES.....	9
INTRODUCCIÓN DE LA NOTACIÓN ALGORÍTMICA A EMPLEAR.....	10

DEFINICIÓN DE ALGORITMO

Los algoritmos se pueden definir como:

“Una secuencia de pasos que son llevados a cabo de forma mecánica y sistemática por un actor que se desenvuelve en un entorno dado para resolver un problema determinado en un tiempo infinito.”

“Una combinación de instrucciones combinadas de forma adecuada para resolver un determinado problema en una cantidad finita de tiempo. Cada instrucción es una indicación y no ambigua.”

Por tanto, como propiedades generales los algoritmos:

- Resuelven un **problema específico**.
- Son llevados a cabo por una **entidad** que trabaja en un **entorno adecuado**.
- Constan de una **serie de pasos** que se llevan a cabo siguiendo una **secuencia marcada**.
- Se aplican de **forma mecánica**.
- Terminan en un **tiempo ~~finito~~ Infinito**

ELEMENTOS PARTICIPANTES EN LA REALIZACIÓN DE UN ALGORITMO

Los elementos que participan en un algoritmo son:

- **Procesador:** Es un ente que es capaz de entender los pasos que componen el algoritmo y llevarlos a cabo.
- **Entorno:** Es el conjunto de materiales necesarios para la ejecución del algoritmo.
- **Acción:** Es un suceso llevado a cabo por el procesador y que modifica el entorno.

Usando el ejemplo de un ordenador tendríamos:

- Un procesador que sería la CPU del propio ordenador.
- Un entorno que serían los datos almacenados en la memoria con los que se va a trabajar.
- Unas acciones que serían las operaciones que el ordenador puede realizar con esos datos (sumas, restas, copiado, ...)

FORMAS DE DESCRIBIR UN ALGORITMO

A. LENGUAJE NATURAL

Es la forma más sencilla. Por ejemplo, el algoritmo para encontrar las raíces de una ecuación de segundo grado sería:

1. Definir los coeficientes de la ecuación de segundo grado:

$$a, b, c$$

2. Determinar el valor del discriminante:

$$b^2 - 4ac$$

3. Si el discriminante es cero sólo hay una solución posible:

$$\frac{-b}{2a}$$

4. Si el discriminante es positivo hay dos soluciones:

$$x = \frac{-b \pm \sqrt{\text{discriminante}}}{2a}$$

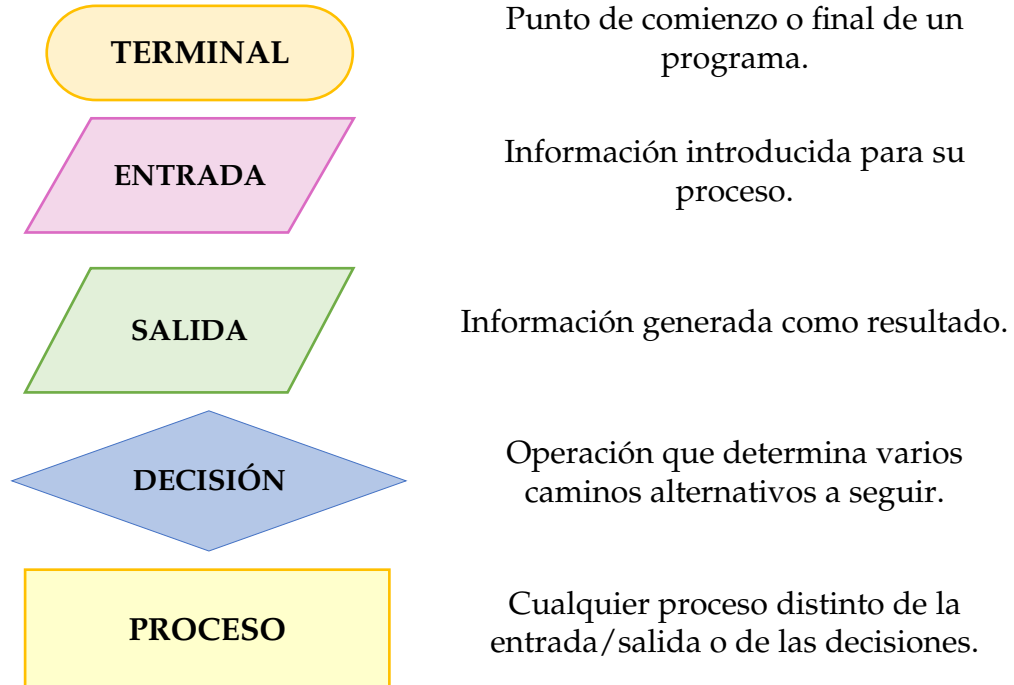
5. Si el discriminante es negativo no hay soluciones reales.

VENTAJAS	INCONVENIENTES
Facilidad de comprensión Cualquier persona que entienda el idioma podría entenderlo sin problemas	No es universal Este ejemplo sólo valdría para hispanohablantes
	Es ambiguo Por tanto, susceptible a errores o malinterpretaciones
	Es demasiado amplio Una misma instrucción puede ser sencilla para una persona pero no para otra.

B. ORGANIGRAMAS

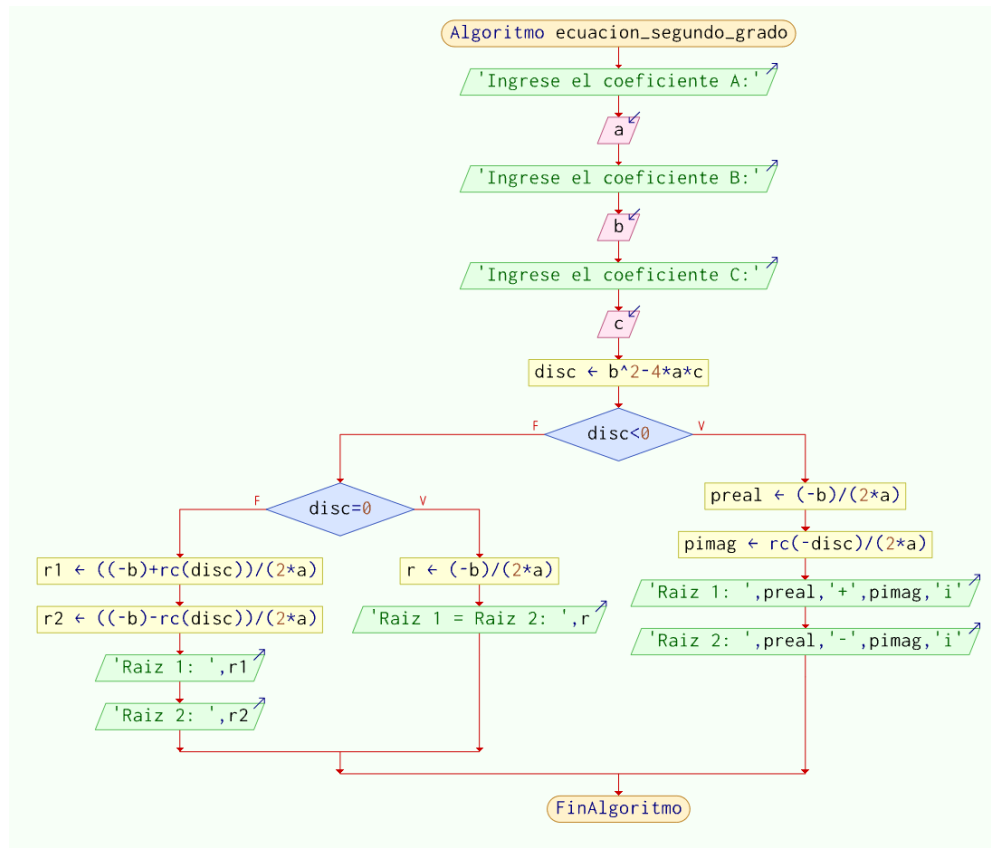
También llamados diagramas de flujo permiten describir los algoritmos de forma gráfica.

Algunos de los símbolos que se emplean son los siguientes:



VENTAJAS	INCONVENIENTES
Son universales	Dificultad de comprensión
Son menos propensos a la ambigüedad	Texto en lenguaje natural
Delimitan mejor los algoritmos	Representación tediosa y a veces confusa
Se aproximan a la forma en que trabaja el ordenador	Un ordenador no es capaz de utilizar una representación como descripción algorítmica.

En el ejemplo de las ecuaciones de las raíces cuadradas el diagrama de flujo sería el siguiente:



C. PSEUDOCÓDIGO

Combina las ventajas del lenguaje natural y de los organigramas sin ninguno de sus problemas. Por tanto:

- Es **fácilmente comprensible** para una persona que lo vea por primera vez.
- Está **bien delimitado**.
- **Elimina las ambigüedades** del lenguaje natural.
- Se representa de **forma compacta**.

El pseudocódigo se suele ver como un subconjunto del lenguaje natural que proporciona un número limitado de operaciones para la construcción de algoritmos. Su finalidad es la comunicación entre seres humanos.

La representación en pseudocódigo del ejemplo de las raíces de ecuaciones de segundo grado es la siguiente:

```

1 // calcula las raices de una ecuacion de segundo grado
2 Proceso ecuacion_segundo_grado
3   // cargar datos
4   Escribir "Ingrese el coeficiente A:"
5   Leer a
6   Escribir "Ingrese el coeficiente B:"
7   Leer b
8   Escribir "Ingrese el coeficiente C:"
9   Leer c
10  // determinar si son reales o imaginarias
11  disc ← b2-4*a*c
12  Si disc<0 Entonces
13    // si son imaginarias
14    preal ← (-b)/(2*a)
15    pimag ← rc(-disc)/(2*a)
16    Escribir "Raiz 1: ",preal,"+",pimag,"i"
17    Escribir "Raiz 2: ",preal,"-",pimag,"i"
18  Sino
19    Si disc=0 Entonces // ver si son iguales o distintas
20      r ← (-b)/(2*a)
21      Escribir "Raiz 1 = Raiz 2: ",r
22    Sino
23      r1 ← ((-b)+rc(disc))/(2*a)
24      r2 ← ((-b)-rc(disc))/(2*a)
25      Escribir "Raiz 1: ",r1
26      Escribir "Raiz 2: ",r2
27    FinSi
28  FinSi
29
30 FinProceso

```

D. LENGUAJES DE PROGRAMACIÓN

Comparten las mismas características que el pseudocódigo, pero no son sólo comprensibles para los humanos sino también para los ordenadores.

Existen múltiples lenguajes de programación. La representación en lenguaje FORTRAN del algoritmo de las raíces de las ecuaciones de segundo grado sería el siguiente:

```

program segundoGrado
print *, 'Deme los coeficientes y resolveré una
ecuación de 2° grado'
print *, '¿Cuánto vale A?'
read *, a
print *, '¿Cuánto vale B?'
read *, b
print *, '¿Cuánto vale C?'
read *, c
discr = b*b-4*a*c
if (discr==0) then
s = -b/(2*a)
print *, 'Sólo hay una solución: ', s
else
if (discr>0) then
s1 = (-b+sqrt(discr))/(2*a)
s2 = (-b-sqrt(discr))/(2*a)

```

```

print *, 'Las soluciones son: ',s1,s2
else
print *, 'No hay soluciones reales'
end if
end if
end

```

DESARROLLO E IMPLEMENTACIÓN DE UN ALGORITMO

Cuando se debe resolver un problema complejo mediante la utilización de un ordenador no se puede codificar directamente un programa en un lenguaje dado (al menos que se quiera incurrir en problemas que veremos más adelante). A pesar de que los lenguajes de programación actuales son cada vez más inteligibles para los humanos, estos no están diseñados para abarcar directamente problemas complejos. Es más, en la mayoría de los casos se trabaja en equipo, debido a la necesidad de presentar la solución al problema en tiempo y forma.

Por tanto, a la hora de solucionar un problema se deben llevar a cabo una serie de fases:

- **Primera fase o “análisis”.** Consiste en el estudio del problema a resolver, pero **no se buscará la forma de resolver el problema** sino comprender la naturaleza de este. ¿Qué datos son necesarios?, ¿En qué consistirá la solución?, ¿Qué errores puede presentar?... Una forma de enfocar el análisis es dividiéndolo en: fase de entrada, proceso y persistencia, salida y condiciones erróneas. Aquí las preguntas a realizar suelen empezar con “QUE”. ¿Qué hara nuestro programa? Cobrar en euros el total de un determinado carrito de un cliente en nuestra tienda online.
- **Segunda fase o “diseño”.** Consiste en **buscar una forma de resolver el problema**. Aquí respondemos a la pregunta “¿Cómo lo haremos?”. La mejor forma de afrontar esta fase es resolver el problema usando distintos niveles de abstracción:
 - o **Nivel de abstracción 1.** En este nivel se plantea el problema usando términos del mismo problema. Ejemplo: aplicaremos una serie de algoritmos para calcular el total a pagar cuando un cliente nos presente su carro de la compra.
 - o **Nivel de abstracción 2.** Se descompone en varios subproblemas expresados en términos del problema y tratando de hacerlos lo más independientes entre sí que sea posible. Siguiendo con el ejemplo anterior: Para calcular el total a pagar hay que calcular el subtotal de cada producto

del carrito. Después sumar las cantidades obtenidas con el proceso anterior para obtener el total.

- o Esta fase 2 se **repite (fase 3, fase 4, fase...n)** para cada **subproblema tantas veces como sea necesario hasta llegar a una descripción que emplee instrucciones sencillas que puedan ser codificadas fácilmente**. En nuestro ejemplo, el subtotal de un producto se calcula multiplicando la cantidad por el precio, aplicando después (sumando) el IVA del producto determinado (IVA multiplicado por el precio y por la cantidad todo dividido entre 100)

En esta fase se pueden emplear diagramas de flujo para representar los diferentes subproblemas y las relaciones entre ellos o **pseudocódigo** para una descripción más “formal” de la resolución de cada subproblema.

CONCEPTOS FUNDAMENTALES

- **Acción:** Suceso llevado a cabo por un procesador que modifica su entorno.
- **Indicador o variable:** El entorno de una acción está compuesto de variables que pueden tener distintos valores a lo largo del tiempo de ejecución del algoritmo. Cada una tendrá un nombre único asignado.
- **Información:** Es la asociación entre una variable (v), un valor (70.5) y la interpretación que se hace (velocidad en km/h).
- **Estado:** Conjunto de valores de las variables en un instante. Por tanto, si una acción modifica su entorno, las acciones hacen pasar el entorno de un estado a otro.
- **Datos y resultados:** Las acciones emplean un tiempo finito para su ejecución. Suponiendo que una acción se inicia en el momento t_0 y termina en t_1 , el estado del sistema en t_0 serían los datos iniciales de la acción mientras que el estado del sistema en t_1 serían los resultados de la acción.
- **Léxico:** Conjunto finito de acciones que el procesador puede entender y ejecutar. Son las “palabras clave del lenguaje”
- **Tipos de datos:** Los datos se clasifican en función de ciertas características importantes con respecto a su naturaleza: números naturales, enteros, reales, complejos, conjuntos de textos, letras, etc. Estos conjuntos pueden estar formados por elementos individuales (el número 6, el texto “hola”) o agregaciones de valores (arrays, tablas, colecciones, etc).
- **Operador:** Son símbolos que representan acciones sencillas. El procesador conoce la forma de realizar operaciones con operadores tales como asignaciones, operaciones matemáticas, comparaciones de valores, etc.

INTRODUCCIÓN DE LA NOTACIÓN ALGORÍTMICA A EMPLEAR

- **Variables:** Los nombres de las variables son identificadores y se expresan usando caracteres alfanuméricos incluyendo el subrayado (barra baja) y excluyendo los blancos (espacios) y caracteres no anglosajones (ñ, vocales acentuadas...). Los identificadores tampoco podrán empezar por un número y en algunos lenguajes las mayúsculas se consideran como caracteres diferentes de las minúsculas (**lenguajes *key sensitive***). Por ejemplo,

- o Identificadores válidos: v, aceleracion, K, v1, b_n, Pot...
- o Identificadores no válidos: 1n (empieza por número), año (incluye un carácter no válido, la ñ), aceleración (incluye un carácter no válido, la ó), p v (incluye un espacio en blanco) ...

El **nombre** de las variables debe ser **representativo del valor que contienen**, de forma que no serán válidas variables con el nombre a o b, pero si dni o nombre_cliente.

Estos nombres van precedidos de un indicador del tipo de datos que contenga. Esto puede variar según políticas de estilo de cada programador, equipo, empresa, pero en general es muy buena idea empezar a programar usando esta nomenclatura. Por ejemplo:

- o **Enteros (int):** iCoeficiente.
- o **Reales (rln):** rNota.
- o **Carácter (chr):** cLetra.
- o **Cadena (str):** strNombre.
- o **Booleandos (bln):** bBandera.
- o **Arrays (arr):** arrNotas1DAW.

- **Tipos de datos:** Se describen en función de las operaciones que admiten.

- o **El tipo entero.** Corresponde a un subconjunto finito de números enteros (el tamaño del rango varía en función del ordenador). Las operaciones soportadas son: suma (+), resta (-), producto (*), división entera (div) y módulo/resto (mod). El resultado de operar dos valores de tipo entero resulta en otro valor de tipo entero con un valor exacto (salvo que resulte un valor fuera del rango).
- o **El tipo real.** Corresponde a un subconjunto de números reales. A diferencia de los valores enteros, los valores reales son valores aproximados susceptibles de errores de redondeo. Admite las mismas operaciones que el tipo entero salvo la división entera y el módulo (la división real

emplea el operador $\wedge$). Además, admiten las funciones reales: raíz cuadrada, exponenciación, potenciación, logaritmo y funciones trigonométricas. Al operar con valores de tipo real resultan valores de tipo real. Se puede operar entre valores de tipo entero y real y aplicar operaciones de tipo real al tipo entero, en todos estos casos, los resultados serán de tipo real.

- o **El tipo lógico o booleano.** Admite dos valores: verdadero y falso; y las operaciones booleanas: y-lógico (and o $\wedge$), o-lógico (or o $\vee$) y no-lógico (not o $\neg$). Hay operadores que, aunque no operan con valores de tipo lógico dan resultados de dicho tipo, son los operadores de comparación.
- o **El tipo caracter.** Comprende el conjunto de caracteres imprimibles:
 - 26 letras del alfabeto latino. 10 dígitos arábigos. Caracteres como símbolos de puntuación.
 - Los subconjuntos de letras y números están ordenados; es decir si un carácter es mayor que 'A' y menor que 'Z' es una letra mayúscula, si es mayor que 'a' y menor que 'z' es minúscula y si es mayor que '0' y menor que '9' es un número.

Este tipo tiene una operación propia, la concatenación de cadenas de caracteres empleando el operador (+).

Todos los tipos admiten las operaciones de comparación:

- o Mayor (>).
- o Menor (<).
- o Mayor o igual (>=).
- o Menor o igual (<=).
- o Igual (=).
- o Distinto (!=).

El resultado de estas comparaciones es siempre un valor de tipo lógico.

- **Literales:** Son expresiones de valor constante de cualquier tipo. Hay literales de tipo entero (por ejemplo, 45, 0 o -17), de tipo real (por ejemplo, 3.141592, 10.5e20 o -5.24e-5), de tipo carácter (por ejemplo, 'E', 'e') y de tipo cadena (por ejemplo, "Esto es una cadena").

En muchas ocasiones es necesario usar un valor literal de forma reiterada, en esos casos puede referirse a dicho valor mediante un identificador (por ejemplo, PI). Para ello se utilizan las denominadas **constantes**, identificadores a los que se les asigna un valor que no puede ser modificado en ningún punto del algoritmo. Se representan con letra mayúscula en todo el nombre.

- **Expresiones:** Son combinaciones de variables, constantes y literales de tipos compatibles entre sí. Estos elementos están combinados mediante operadores válidos. Es posible tener expresiones de cualquier tipo de dato, por ejemplo:

$2 * PI * r$
`intVelocidad * intTiempo`
`(intCont>5) and (intCont<10)`
`'Hola' + 'mundo'`

- **Asignación:** El operador de asignación (β) se emplea según la notación siguiente:

`izquierda  $\beta$  derecha`

El elemento situado a la izquierda es una variable a la que se le asigna el valor de la expresión de la derecha. Ambos elementos deben ser del mismo tipo. En el caso de Java, lenguaje que usaremos en el curso, la asignación es un carácter '='. Ejemplo:

`iAge = 18;`
`//El valor 18 se almacena en la variable iAge`

- **Entrada/salida:** Los algoritmos necesitan de una interacción usuario-algoritmo, por ejemplo, es posible que el usuario deba indicar una serie de valores (como los coeficientes de una ecuación de segundo grado) o que el algoritmo deba proporcionar unos resultados al usuario (las posibles soluciones de la ecuación). Las acciones de entrada y salida son: **leer** y **escribir**.

El formato de **leer** es: `leer variable`. El resultado de la acción es la lectura de un valor por teclado que será asignado a **variable**.

El formato de **escribir** es: `escribir expresión` (expresión puede ser de cualquier tipo). El resultado de la acción es la visualización en un dispositivo adecuado de la **expresión**.